Document number	P3326R0
Date	2024-06-13
Reply-to	Jarrad J. Waterloo <descender76 at gmail dot com>
Audience	Library Evolution Working Group (LEWG)

favor ease of use

Table of contents

Abstract

`optional<&>` [1] is a spectacularly `safe by default` type. With only small additions to both `optional<T>` and `optional<T&>` [1:1], library writers will be able to make it even easier to use by allowing temporaries to be used in safe scenarios.

Motivational Example

p2988r5 [1:2]

given

```
optional<int&> dangler(optional<const int&> other,
                       optional<int&> left,
                       optional<int&> right)
{
    if(random_bool())
    {
        return left;
    }
    else
    {
        return right;
    }
}
```

usage

```
int i = 42;
optional<int&> oi1{i};
optional<int&> oi2 = dangler(oi1, oi1, oi1);
optional<int&> oi3 = dangler(42/* unnecessary error */, oi1, oi1);
```

this proposal

given

```
// just added ", favors::ease" to the first parameter
// since 'other' nor any component of 'other' is returned
// in other words, the return is only dependent upon left and right
optional<int&> dangler(optional<const int&, favors::ease> other,
                       optional<int&> left,
                       optional<int&> right)
{
    if(random_bool())
    {
        return left;
    }
    else
    {
        return right;
    }
}
```

usage

```
int i = 42;
optional<int&> oi1{i};
optional<int&> oi2 = dangler(oi1, oi1, oi1);
optional<int&> oi3 = dangler(42/* ok */, oi1, oi1);
```

Requested Changes

1. Add the `favors` enumeration

```
enum class favors
{
    safety,
    ease
};
```

2. Add the default value of `favors::safety` to the original `optional` template.

```
template <class T, favors favor = favors::safety>
class optional {
};
```

3. Add the `favor` parameter to the `std::optional<&>` [1:3] specialization.
4. Revise the `std::optional<&>` [1:4] constructor by requiring `favors::safety` .
5. Add a constructor that takes a lvalue reference and requiring `favors::ease` .

```
template <class T, favors favor/* = favors::safety*/>
class /*std::*/optional<T&, favor> {

// ...

        template <class U = T>
            requires(!detail::is_optional<std::decay_t<U>>::value)
        constexpr explicit(!std::is_convertible_v<U, T>) optional(U&& u) noexcept
            : value_(std::addressof(u)) {
            static_assert(
                std::is_constructible_v<std::add_lvalue_reference_t<T>, U>
                & favor == favors::safety,
                "Must be able to bind U to T&");
            static_assert(std::is_lvalue_reference<U>::value
                & favor == favors::safety,
                "U must be an lvalue");
        }

        constexpr optional(T& t) noexcept requires (favor == favors::ease)
            : value_{std::addressof(t)} {}

// ...

}
```

`favors::safety` vs `favors::ease` usage

favors::safety (i.e. the default)

- local variables
- return parameters just like `p2748r5` [2]
- member variables that are dependent upon constructor arguments
- input parameters that will be returned in whole or in part

favors::ease

- input parameters of functions that return void
- input parameters of functions that return values
- input parameters that will NOT be returned in whole or in part

Should've, Could've, Would've

The proposed functionality could be of benefit in other pure reference types. While that is out of scope of this proposal, it is worth mentioning the current impediments.

span

Span is not as `safe by default` as `std::optional<&>` [1:5] since it allows binding to a temporary by default.

```
std::string_view sv1 = "42"s;// should be error
```

function_ref

It can be difficult to implement as parameter packs and default parameter values both live at the end of the template parameter list.

reference_wrapper

`reference_wrapper` is the best existing pure reference type that could benefit from this enhancement. Its safety is comparable to that of `std::optional<&>` [1:6].
`reference_wrapper` would need to be reimplemented and respecified before enhancing. Doing such would result in a safer replacement for `&` itself.

Summary

This proposal identifies with Arthur O'Dwyer's article `Value category is not lifetime` [3]. While `safety by default` is paramount, requiring that all libraries that use `std::optional<&>` [1:7] be needlessly difficult to use is detrimental to the clarity of our programs.

References

1. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2988r5.pdf (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2988r5.pdf) ↩️ ↩️ ↩️ ↩️ ↩️ ↩️ ↩️ ↩️

2. https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2748r5.html (https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2024/p2748r5.html) ↩️

3. https://quuxplusone.github.io/blog/2019/03/11/value-category-is-not-lifetime/
   (https://quuxplusone.github.io/blog/2019/03/11/value-category-is-not-lifetime/) ↩️